AI Recruitment Portal

Database Schema Documentation

Technology Stack: FastAPI | SQLAlchemy ORM | PostgreSQL

Prepared by: Syeda Saba Fathima

Date: July 2026

Schema Overview

This database consists of 8 tables designed to support the full AI Recruitment Portal workflow — from recruiter authentication, through resume upload and AI-powered parsing, to AI-driven candidate evaluation, interview question generation, and an AI chat assistant. All tables are defined using SQLAlchemy ORM models and mapped to PostgreSQL; no raw SQL is used for schema creation or data access.

#	Table	Purpose
1	users	Recruiter accounts and authentication
2	candidates	Core candidate records and pipeline status
3	resumes	Uploaded resume files and AI-extracted content
4	job_descriptions	Job postings entered by recruiters
5	evaluations	AI match scoring plus human ratings per candidate/job pair
6	interview_questions	AI-generated technical, scenario, and coding questions
7	chat_logs	AI Chat Assistant conversation history
8	activity_logs	System-wide audit trail of user actions

1. users

Stores recruiter account credentials and profile information. Every recruiter must have a row here to log in and use the portal.

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
username	String	Unique	Required, used for login
email	String	Unique	Required, used for login
hashed_password	String	Required	Bcrypt-hashed, never stored in plain text
full_name	String	Optional	Recruiter's display name
role	String	Default: recruiter	Reserved for future role-based access
created_at	DateTime	Auto-set	Set automatically by PostgreSQL on insert

Relationships

• One user has many **activity_logs** (1-to-many) — every login, upload, and status change is tracked per user.
• One user has many **chat_logs** (1-to-many) — full AI Chat Assistant history per recruiter.

2. candidates

The core table representing each person who has applied. Created automatically when a resume is uploaded and parsed.

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
name	String	Required	Extracted from resume by AI parser
email	String	Optional	Extracted from resume
phone	String	Optional	Extracted from resume
experience_years	Float	Default: 0	Allows decimals, e.g. 2.5 years
education	String	Optional	Extracted from resume
status	String	Default: pending	pending / shortlisted / rejected — drives dashboard counts
created_at	DateTime	Auto-set	When candidate record was first created
updated_at	DateTime	Auto-updates	Refreshed automatically on every edit

Relationships

• One candidate has exactly one **resume** (1-to-1) — enforced via unique constraint on resumes.candidate_id.
• One candidate has many **evaluations** (1-to-many) — one per job description they're assessed against.
• One candidate has many **interview_questions** (1-to-many) — generated per job description.

3. resumes

Stores the uploaded file reference and all AI-extracted content from a candidate's resume (PDF or DOCX).

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
candidate_id	Integer	FK, Unique	References candidates.id — one resume per candidate
file_name	String	Required	Original uploaded filename
file_path	String	Required	Server storage location of the file
file_type	String	Optional	pdf or docx
raw_text	Text	Optional	Full extracted text content
skills	Text	Optional	AI-extracted skills list
certifications	Text	Optional	AI-extracted certifications
projects	Text	Optional	AI-extracted project descriptions
uploaded_at	DateTime	Auto-set	When the file was uploaded

Relationships

• Belongs to exactly one **candidate** (1-to-1, reverse of candidates.resume).

4. job_descriptions

Stores job postings entered by recruiters, used as the comparison basis for AI resume analysis and interview question generation.

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
title	String	Required	e.g. "Senior Python Developer"
description	Text	Required	Full job description text
required_skills	Text	Optional	Optionally separated skills list
created_at	DateTime	Auto-set	When the job description was entered

Relationships

• One job description has many **evaluations** (1-to-many) — one per candidate assessed against it.
• One job description has many **interview_questions** (1-to-many) — generated per candidate.

5. evaluations

Junction table linking a candidate to a specific job description, storing both AI-generated match analysis and human recruiter/technical/HR ratings.

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
candidate_id	Integer	FK	References candidates.id
job_description_id	Integer	FK	References job_descriptions.id
match_score	Float	Optional	AI-generated percentage match (0-100)
matching_skills	Text	Optional	AI-identified overlapping skills
missing_skills	Text	Optional	AI-identified skill gaps
experience_gap	String	Optional	Short AI summary, e.g. "Meets requirement"
ai_recommendation	Text	Optional	AI-generated recommendation paragraph
recruiter_rating	Float	Optional	Human rating from recruiter
technical_rating	Float	Optional	Human rating from technical interviewer
hr_rating	Float	Optional	Human rating from HR
final_recommendation	String	Optional	e.g. "Hire" / "Reject"
created_at	DateTime	Auto-set	When this evaluation was created

Relationships

• Belongs to one **candidate** (many-to-1).
• Belongs to one **job_description** (many-to-1).
• A candidate can have multiple evaluations, one per job description considered.

6. interview_questions

Stores each AI-generated interview question as an individual row, categorized by type, for a specific candidate and job description pair.

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
candidate_id	Integer	FK	References candidates.id
job_description_id	Integer	FK	References job_descriptions.id
question_text	Text	Required	The full question content
question_type	String	Optional	technical / scenario / coding
created_at	DateTime	Auto-set	When the question was generated

Relationships

- Belongs to one **candidate** (many-to-1).
- Belongs to one **job_description** (many-to-1).
- Typically 10 rows generated per candidate/job pair: 5 technical, 3 scenario, 2 coding.

7. chat_logs

Stores the full conversation history between a recruiter and the AI Chat Assistant.

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
user_id	Integer	FK	References users.id — the recruiter who sent the message
user_message	Text	Required	What the recruiter typed
ai_response	Text	Optional	The AI assistant's reply
created_at	DateTime	Auto-set	When this exchange occurred

Relationships

• Belongs to one **user** (many-to-1) — one recruiter can have many chat log entries.

8. activity_logs

System-wide audit trail recording every significant action taken in the portal — logins, resume uploads, resume analysis runs, and candidate status changes.

Column	Type	Constraint	Notes
id	Integer	PK	Auto-incrementing unique identifier
user_id	Integer	FK	References users.id — who performed the action
action_type	String	Required	login / resume_upload / resume_analysis / candidate_status_update
description	String	Optional	Human-readable detail of the action
created_at	DateTime	Auto-set	Exact timestamp of the action

Relationships

• Belongs to one **user** (many-to-1) — one recruiter can have many activity log entries.

Entity Relationship Diagram

The diagram below shows all 8 tables and how they connect. PK denotes a Primary Key, FK denotes a Foreign Key. Blue connectors represent one-to-many relationships; the gold connector represents the one-to-one relationship between candidates and resumes.